

REACT LAB SERIES

Practical React

Lab 1 → Lab 2 → Lab 3

A progressive three-lab series — each lab extends the previous. Build a complete Student Dashboard application from scratch.

LAB 01 Foundation UI Components, Props, Styling	LAB 02 State & Effects useState, useEffect, Interactivity	LAB 03 Context & Forms useContext, Form Validation
---	---	--

LAB 01

React Components, Props & Custom Styling

Build the static UI shell of a Student Dashboard

OBJECTIVE

Create a static Student Dashboard with reusable React components. Students will learn component decomposition, passing data through props, and applying a consistent design system using custom CSS or CSS Modules.

TOPICS COVERED

- ◆ React Components — functional components and JSX
- ◆ Reusable Components — building generic, configurable UI pieces
- ◆ Props — passing and validating data between components
- ◆ Styling Components — custom CSS with CSS variables (design tokens)

TASKS

1. **StudentCard Component** — Create a StudentCard reusable component that accepts props: name, id, avatar, gpa, and major. Render at least 4 student cards on the page.
2. **CourseTag Component** — Build a CourseTag component that accepts a courseName and color prop. It should render a styled pill/badge. Use it inside StudentCard to show enrolled courses.
3. **StatBadge Component** — Create a StatBadge component that shows a label-value pair (e.g., GPA: 3.8, Credits: 90). Reuse it in at least two different places in the dashboard.
4. **DashboardHeader & Styling** — Build a DashboardHeader component with a title, tagline, and navigation bar. Style the entire application using a custom CSS file — define at least 5 CSS custom properties (variables) for colors, font sizes, and spacing.
5. **App Composition & PropTypes** — Compose all components inside a main App component and ensure proper parent-to-child prop flow. Add PropTypes validation for all components.

LAB 02

State Management, Side Effects & Interactivity

Make the Student Dashboard interactive with dynamic data

↑ **Extends from previous lab** — Continue from Lab 01. Add state and effects to existing components. Do not start from scratch.

OBJECTIVE

Extend the Lab 01 dashboard by introducing interactivity through `useState` and lifecycle behaviors through `useEffect`. Students will add search, filtering, and data simulation features to make the dashboard dynamic.

TOPICS COVERED

- ◆ `useState` — managing local component state
- ◆ `useEffect` — side effects, data fetching simulation, and cleanup
- ◆ Reusable Components — `SearchBar` and `SortControls`
- ◆ Props & state lifting — sharing state between parent and children

TASKS

1. **Simulated API Fetch** — Replace hardcoded student data in App with a `useState` hook. Simulate an API fetch using `useEffect` with `setTimeout` to load student data after 1.5 seconds. Show a loading spinner during data fetch.
2. **Live Search** — Add a live search bar (new `SearchBar` reusable component) that filters the student list in real-time by name or major using `useState` for the query and derived filtering logic.
3. **Favorite Toggle & State Lifting** — Add a Favorite toggle button inside each `StudentCard`. Use `useState` inside the card to track the toggle state (icon change, color change). Lift state to App to show a count of total favorites in `DashboardHeader`.
4. **Dynamic Document Title** — Use `useEffect` to update the browser tab title (`document.title`) to reflect the number of students currently displayed (e.g., Dashboard — 4 Students). Update it whenever the search filters change.
5. **Sort Controls** — Create a `SortControls` reusable component with buttons to sort students by Name (A–Z), GPA (high to low), and Default order. Store sort preference in `useState` and apply the sort before rendering the student list.

LAB 03

Global State with Context & Form Validation

Add a new student registration flow with global context

↑ **Extends from previous lab** — Continue from Lab 02. Integrate Context API and a validated form. Same project, new layer.

OBJECTIVE

Extend the Lab 02 dashboard by adding global theme management via the Context API and a fully validated Add Student registration form. By the end, students will have a complete, full-featured React dashboard application built progressively across all three labs.

TOPICS COVERED

- ◆ `createContext` & `useContext` — creating and consuming context
- ◆ Context Provider pattern — `ThemeContext` and `StudentContext`
- ◆ Form Validation — inline errors, field rules, and submission handling
- ◆ `useState` & `useEffect` — applied in form and notification flows
- ◆ `localStorage` persistence — rehydrating state on page load

TASKS

- 1. ThemeContext — Light/Dark Mode** — Create a `ThemeContext` using `createContext` and a `ThemeProvider` component. Support light and dark themes. Wrap the entire app in the provider. Add a toggle button in `DashboardHeader` that uses `useContext(ThemeContext)` to switch themes — all components must respond to the theme change.
- 2. StudentContext & Refactoring** — Create a `StudentContext` that holds the global list of students, the search query, sort preference, and favorites. Move all related state from `App` into this context. Refactor `SearchBar`, `SortControls`, and `StudentCard` to consume state via `useContext` instead of prop drilling.
- 3. AddStudentForm with Validation** — Build an `AddStudentForm` component with fields: Full Name, Student ID, Major, GPA, and Courses (comma-separated). Implement validation: name must be non-empty, ID must be unique and numeric, GPA must be between 0–4.0, major is required. Show inline error messages below each invalid field.
- 4. Form Submission & Success Notification** — On successful form submission, add the new student to `StudentContext` so the dashboard immediately reflects the addition without a page reload. Reset the form after submission and show a success notification using `useEffect` that auto-dismisses after 3 seconds.
- 5. Remove Student & localStorage Persistence** — Add a Remove Student button to each `StudentCard` that removes the student from context. Use `useEffect` to persist the student list to `localStorage` and rehydrate it on app load, so data survives a page refresh.